Software Architecture and Design Specification

Project: Bank Management System

Field	Details
Version	1.0
Authors	SRIKANTH V REDDY – PES2UG24CS519 T. MAHALAKSHMI – PES2UG24CS558 SUDHANWA – PES2UG24CS531 SRUJAN – PES2UG24CS526
Date	01-10-2026
Status	Draft

Revision History

Version	Date	Author	Change Summary
1.0	01-10-2026	BMS Team	Initial architecture and design specification

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Bank Management System (BMS). It describes the system structure, major components, architecture pattern, technology stack, data stores, security architecture, design flows, API interfaces, error handling, UX design, and traceability to the BMS SRS.

1.2 Scope

The design covers customer and staff authentication, customer and account management, balance inquiry, deposits, withdrawals, fund transfers, beneficiary management, transaction history, statements, loan management, notifications, administration, reporting, and audit logging.

1.3 Audience

Developers, QA engineers, security reviewers, project evaluators, system administrators, and maintenance teams.

1.4 Definitions

BMS, SRS, STP, RTM, API, RBAC, OTP, KYC, TLS, ADR, UAT.

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including component architecture, architecture rationale, security architecture, sequence diagrams, API design, error handling, UX design, and traceability to SRS requirements.

2.2 Related Documents

- Bank Management System SRS v1.0
- Bank Management System Software Test Plan v1.0
- Requirements Traceability Matrix (RTM)

3. Architecture

3.1 Goals & Constraints

- Goals: secure, reliable, modular, maintainable, auditable banking operations with good response time.
- Constraints: protection of sensitive data, RBAC, transaction consistency, availability of dependent services, and controlled access to financial operations.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Customers	Security, correct balances, transaction confirmation, usability, availability
Bank Staff	Efficient account/loan operations, accurate records, controlled permissions
Administrators	Maintainability, configuration, access control, monitoring
Auditors	Traceability, audit logs, reports, data integrity
Developers	Modularity, testability, maintainability, clear interfaces

3.3 Component (UML) Diagram

The BMS component architecture consists of the following logical components: Customer/Staff UI, API Gateway, Authentication Service, Account Service, Transaction Service, Loan Service, Notification Service, Reporting & Audit, and Database.

Component	Responsibility
Customer / Staff UI	Dashboards, forms, account, transaction, loan and administration screens
API Gateway	HTTPS boundary and request routing
Authentication Service	Login, OTP, sessions and RBAC
Account Service	Customer/KYC and account lifecycle
Transaction Service	Deposits, withdrawals, transfers and transaction records
Loan Service	Applications, approvals, schedules and repayments
Notification Service	Configured banking/security alerts
Reporting & Audit	Statements, reports and audit records
Database	Persistent customer, account, transaction, loan, user and audit data

3.4 Component Descriptions

- UI handles role-specific interaction.
- API Gateway provides the controlled application boundary.
- Authentication Service validates identity and authorization.
- Account Service manages customer and account data.
- Transaction Service performs financial operations.
- Loan Service manages loan workflows.
- Notification Service sends configured alerts.
- Reporting & Audit provides reports and traceability.
- Database stores persistent system data.

3.5 Chosen Architecture Pattern and Rationale

A layered service-oriented architecture is selected. Presentation, application/business logic, and data access are separated, while banking responsibilities are grouped into logical services. This improves modularity, testability and maintainability. A fully distributed microservices deployment would add unnecessary operational complexity for the academic project scope.

3.6 Technology Stack & Data Stores

Layer	Technology / Store
Frontend	HTML5, CSS, JavaScript or React
Backend/API	Node.js/Express or equivalent REST API framework
Database	MySQL or another relational DBMS
Authentication	Secure password hashing, sessions/tokens, OTP where configured
Communication	HTTPS with TLS 1.2+
Testing/Tools	Git, Postman, Selenium/equivalent, JMeter where required
Data	Customers, accounts, transactions, beneficiaries, loans, users/roles and audit records

3.7 Risks & Mitigations

Risk	Mitigation
Network/API failure	Clear transaction states, controlled retries and reconciliation
Duplicate transaction request	Unique transaction IDs and atomic DB operations
Unauthorized access	Authentication and RBAC
Data loss/corruption	Constraints, backups and recovery procedures
External service failure	Sandbox/stub services and controlled retry handling

3.8 Traceability to Requirements

SRS Requirement	Architecture Component
BMS-F-001/F-002 – Authentication/OTP	Authentication Service + API Gateway
BMS-F-004/F-005 – Account management	Account Service + Database
BMS-F-007/F-008 – Deposit/Withdrawal	Transaction Service + Database
BMS-F-009/F-010 – Transfer/Beneficiary	Transaction + Authentication Services
BMS-F-013 to F-016 – Loan	Loan Service + Database
BMS-F-017 – Notifications	Notification Service
BMS-F-018/F-019 – Admin/Reports	Administration + Reporting
BMS-F-020 – Audit	Reporting & Audit + Database
BMS-SR-001 to SR-005 – Security	API Gateway + Authentication + RBAC + Audit

3.9 Security Architecture

- Spoofing: authentication, OTP and RBAC.
- Tampering: authorization, validation, DB constraints and audit trails.
- Information disclosure: TLS and controlled data access.
- DoS: rate limiting, validation, timeouts and monitoring.
- Elevation of privilege: RBAC by user role.
- Auditability: user, timestamp, action, outcome and reference recorded for sensitive operations.

4. Design

4.1 Design Overview

User requests enter through the UI and API boundary, authentication and authorization are checked, the appropriate service applies business rules, and the database stores the resulting state. Financial operations use controlled transaction processing so debit and credit changes are committed consistently.

4.2 UML Sequence Diagrams

Flow	Actors / Components	Main Steps
Fund Transfer	Customer → UI → Authentication → Transaction Service → Database	Login/OTP → enter transfer → validate accounts/balance → debit/credit → confirmation
Loan Application	Customer → UI → Loan Service → Staff → Database	Submit application → validate/store → staff review → approve/reject → update status → notify customer

4.3 API Design

Endpoint	Method	Request	Response / Errors
/auth/login	POST	{username,password}	{token,role}; 401 Invalid credentials
/auth/verify-otp	POST	{userId,otp}	{status}; 401 Invalid/expired OTP
/accounts/balance	GET	accountId	{balance}; 403 Unauthorized
/transactions/transfer	POST	{sourceAccount,destinationAccount,amount}	{transactionId,status}; 400 Invalid data, 402 Insufficient funds
/loans/apply	POST	{customerId,amount,tenure,details}	{applicationId,status}; 400 Validation error
/reports/statement	GET	accountId,fromDate,toDate	Statement; 403 Unauthorized

4.4 Error Handling, Logging & Monitoring

- Standardized API errors for validation, authentication, authorization and transaction failures.
- Passwords and OTPs shall never be logged.
- Financial failures shall not create partial debit/credit updates.
- Audit logs shall contain user, timestamp, action, outcome and relevant reference.
- Monitor API errors, transaction failures, availability, authentication failures and database health.

4.5 UX Design

The BMS UI shall provide role-specific dashboards, clear navigation, readable transaction summaries, validation messages and confirmation for financial actions. Customer screens prioritize balances, transactions, transfers, statements and loans; staff screens provide controlled operational functions; administrators receive user/role management, reports and monitoring.

4.6 Open Issues & Next Steps

- Finalize the exact frontend/backend framework selected by the team.
- Finalize database schema and indexing.
- Define detailed API token/session strategy.
- Complete UI wireframes and accessibility review.
- Consider future biometric authentication, mobile integration or fraud detection if scope permits.

5. Appendices

5.1 Glossary

BMS, SRS, STP, RTM, API, RBAC, OTP, KYC, TLS, ADR, UAT.

5.2 References

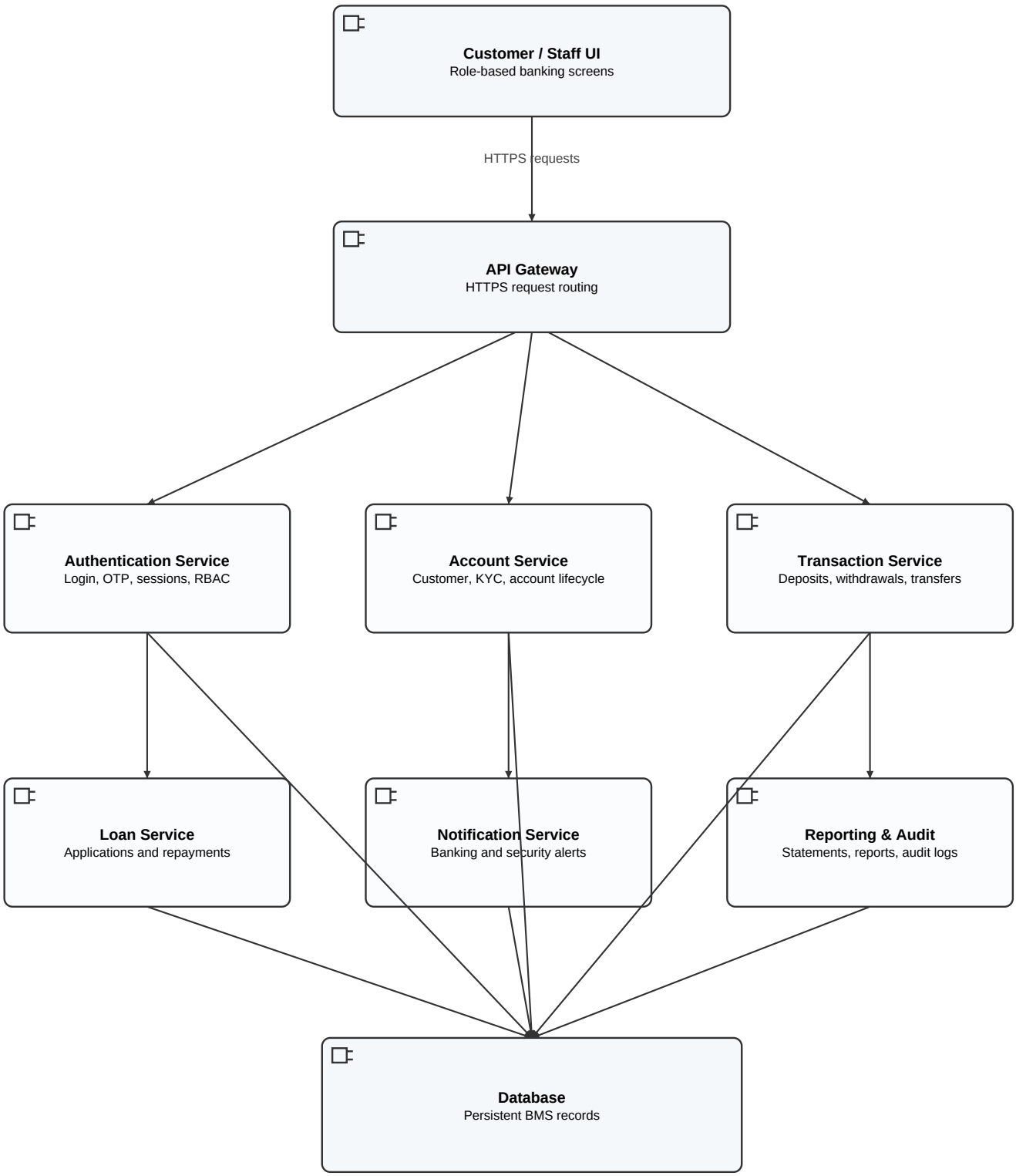
Bank Management System SRS v1.0; Bank Management System Software Test Plan v1.0; provided Software Architecture and Design Specification template.

5.3 Tools

draw.io / diagrams.net, PlantUML or equivalent UML tools, Postman, Git, database tools, and JMeter where performance testing is required.

3.3 UML Component Diagram

The diagram shows the main BMS components and their dependencies.



Component interaction

The UI communicates with the API Gateway, which routes requests to the relevant BMS service.

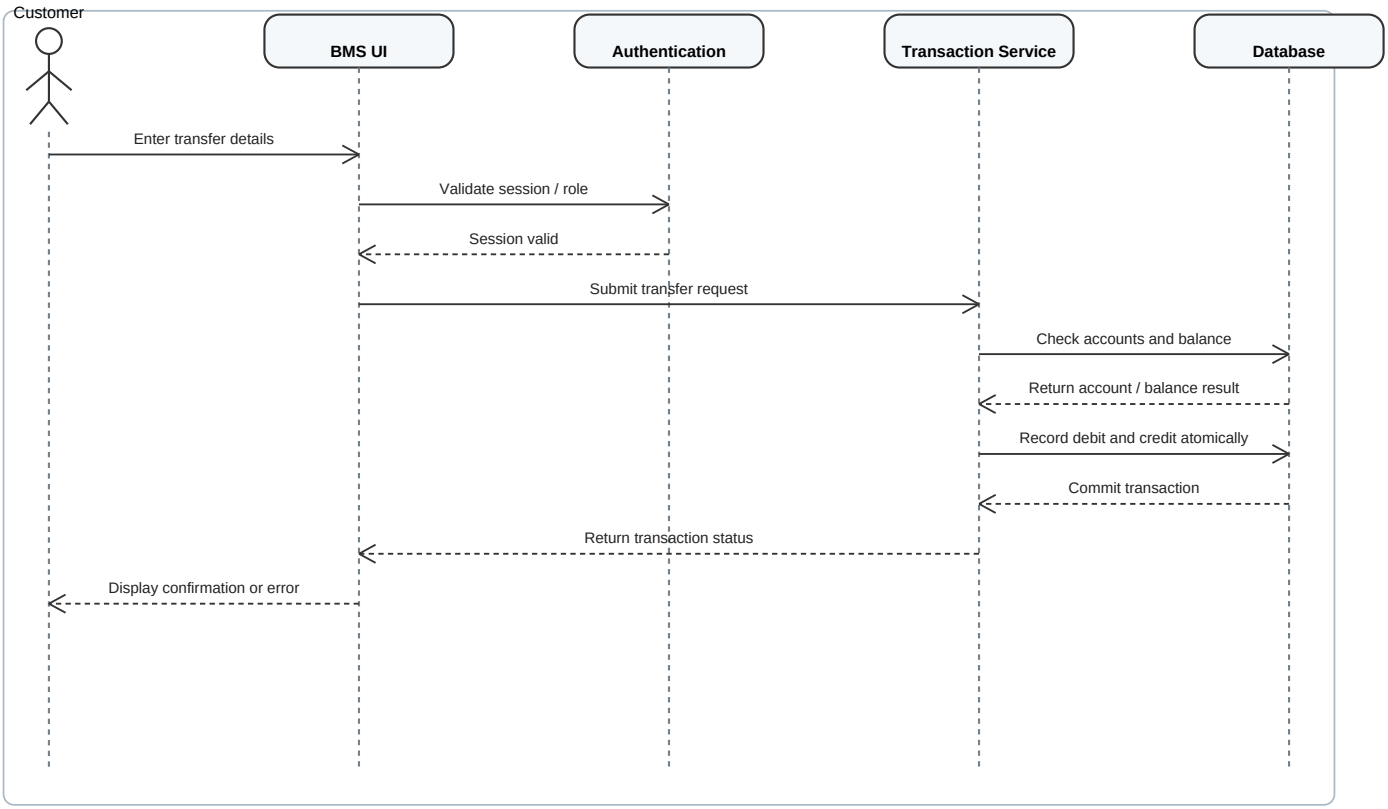
Services use the Database for persistent customer, account, transaction, loan and audit records.

Authentication and RBAC control access to role-specific banking operations.

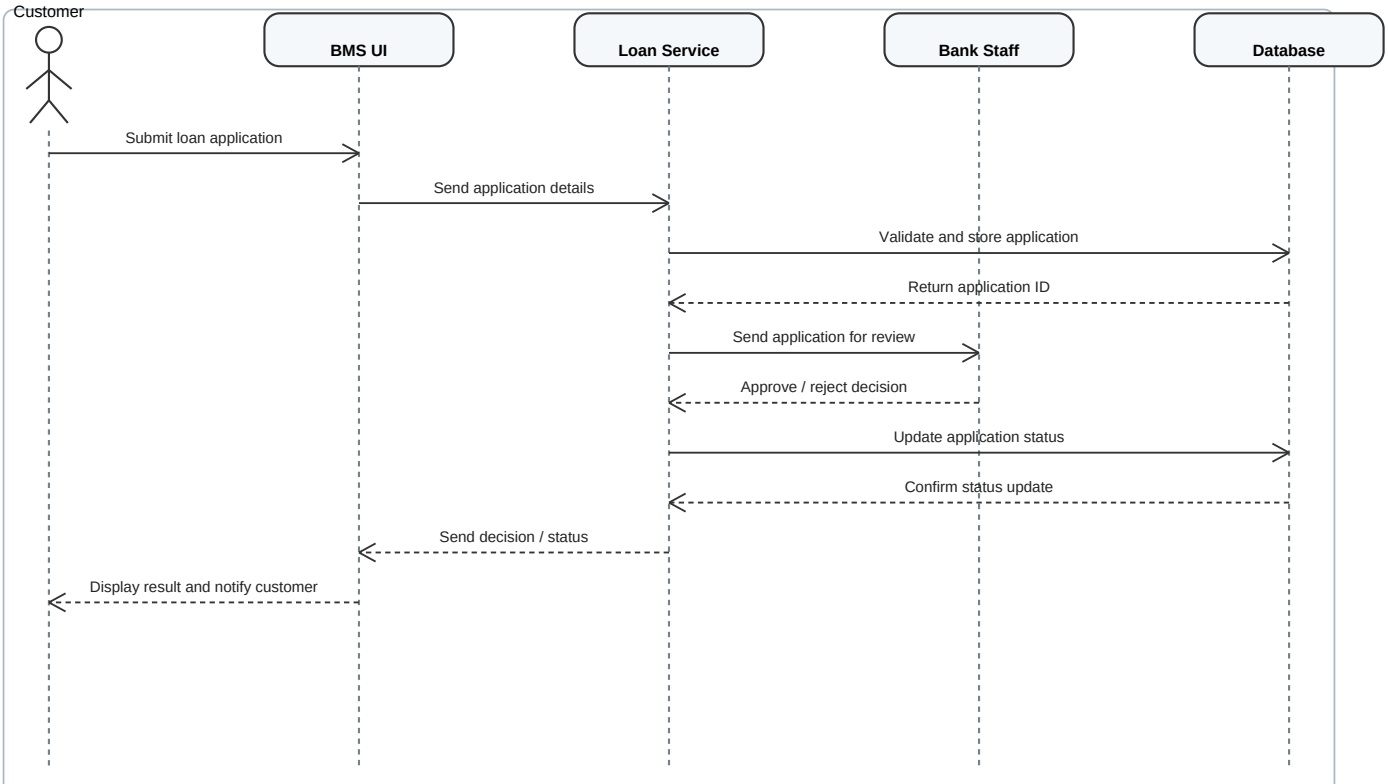
4.2 UML Sequence Diagrams

Message flow for the fund transfer and loan application workflows.

4.2.1 Fund Transfer



4.2.2 Loan Application



Component responsibilities

- UI sends authenticated requests through the API Gateway.
- The gateway routes requests to the relevant banking service.
- Services use the Database for persistent customer, account, transaction, loan and audit data.
- Authentication and RBAC protect access to role-specific operations.